

Part01 – Questions & Answers

Q1: Why is it better to code against an interface rather than a concrete class?

A: Coding against an interface (like `IVehicle`) rather than a concrete class (like `Car`) means your code only depends on the set of behaviors the interface promises, not on any specific implementation. This gives loose coupling — calling code doesn't care which concrete class it's actually working with — and flexibility to swap or add implementations (replace `Car` with `Bike`, or add `Truck` later) without touching any code that already depends on the interface. It also makes unit testing easier, since a mock object implementing the same interface can be substituted for the real one, and it follows the Open/Closed Principle: the system stays open for extension through new implementing classes while the existing dependent code remains closed for modification.

Q2: When should you prefer an abstract class over an interface?

A: Prefer an abstract class when you need to share actual implementation code — not just a contract — among closely related derived classes, such as the non-abstract `Display()` method in `Shape` that every derived type reuses as-is. It's also the better choice when the derived classes need to share common state (fields), and when there's a clear IS-A hierarchical relationship among a family of closely related types. Since a class can only inherit from one abstract class, this fits naturally when the types genuinely belong to a single lineage. Use an interface instead when you just need to describe a capability or contract that potentially unrelated classes can implement (like `Comparable` or `Walkable`), especially when a class might need to support several independent behaviors at the same time — something only possible through interfaces, since a class can implement many of them.

Q3: How does implementing `Comparable` improve flexibility in sorting?

A: Implementing `Comparable<T>` lets a custom type define its own natural ordering logic directly inside the class itself, through the `CompareTo` method. Once that's done, built-in .NET sorting utilities like `Array.Sort()` or `List<T>.Sort()` can sort a collection of these objects automatically, without any custom sorting code being written at every call site. This improves flexibility because the sorting rule lives in exactly one place — the class itself — instead of being duplicated everywhere sorting happens; any built-in or third-party method expecting an `Comparable` object works with the type immediately; and if the sorting criteria ever need to change, only the `CompareTo` method needs to be updated, and every piece of code that sorts the type benefits automatically.

Q4: What is the primary purpose of a copy constructor in C#?

A: The primary purpose of a copy constructor is to create a new, fully independent object that starts out with the same data as an existing object of the same type, so that later changes to one object never affect the other. This matters most for reference types (classes), where a plain assignment (`Student s2 = s1;`) only copies the reference — both variables end up pointing to the exact same object, so a change through one is visible through the other. It becomes especially important when a class has nested reference-type fields (like an `Address` inside a `Student`): only a deliberately written copy constructor can ensure that nested object is also copied into a brand-new instance, rather than simply being shared between the original and the 'copy'.

Q5: How does explicit interface implementation help in resolving naming conflicts?

A: Explicit interface implementation — writing `void IWalkable.Walk()` instead of `public void Walk()` — lets a class provide a separate, distinct implementation for a member that would otherwise clash with another member of the same name, whether that's one of the class's own members or a member from a different interface with the same signature. The explicitly implemented member is no longer accessible directly through the class type; it only becomes visible and callable through a reference of that specific interface type. This resolves naming conflicts by keeping the two implementations fully independent instead of forcing them to share a single method body, and it also lets a class 'hide' an interface member from ordinary users of the concrete class, exposing it only when the object is deliberately treated as that interface.

Q6: What is the key difference between encapsulation in structs and classes?

A: The core idea of encapsulation — hiding internal fields and exposing controlled access through properties or methods — works identically in both structs and classes. The practical difference comes from their different copy semantics. In a class, the encapsulated data belongs to a single object on the heap that every reference points to, so any legitimate change made through the exposed properties/methods mutates that one shared object, and every reference sees the update. In a struct, because the entire value is copied on every assignment or method call, the encapsulated data inside one copy is completely independent of any other copy — changing a property on one struct instance never affects a different copy of that same struct sitting elsewhere, even though both copies expose the exact same encapsulated property. So encapsulation in a struct protects the state of just that one specific copy, while encapsulation in a class protects the state of the single object every reference actually shares.

Q7: What is abstraction as a guideline, what's its relation with encapsulation?

A: Abstraction as a guideline means designing a type around WHAT it does — its essential behavior and capabilities — rather than HOW it does it internally, and deliberately hiding unnecessary implementation complexity behind a simple, focused public interface (for example, `IShape` only exposes `Area` and `Draw()`, without revealing how the area is actually calculated). Its relation with encapsulation is that they work together but address different concerns: encapsulation is the mechanism — using access modifiers to hide fields and expose controlled access — while abstraction is the design principle that decides WHAT should be exposed and WHAT should stay hidden in the first place. In practice, encapsulation is how abstraction gets implemented in code: you cannot achieve good abstraction without encapsulation to actually hide the irrelevant details, and encapsulation without a clear abstraction goal in mind just becomes arbitrary field-hiding with no real design purpose behind it.

Q8: How do default interface implementations affect backward compatibility in C#?

A: Default interface methods (introduced in C# 8.0) let an interface author add a new member to an already-published interface without breaking any existing class that implements it. Before C# 8.0, adding any new member to an interface was a breaking change: every class that already implemented that interface would fail to compile until it added an implementation for the new member. With a default implementation supplied directly in the interface, existing implementing classes simply inherit that default automatically and keep compiling and working exactly as before, while new or updated classes remain free to override the default with their own custom behavior. This lets library and API authors evolve their interfaces over time while staying backward compatible with all the code that already depends on them.

Q9: How does constructor overloading improve class usability?

A: Constructor overloading lets a class be instantiated in several convenient ways depending on exactly what data the caller has available at that moment — for example `Book()`, `Book(title)`, and `Book(title, author)` — instead of forcing every caller to always supply every piece of data even when some of it isn't known or relevant yet. This improves usability because the class becomes more approachable and flexible for different situations, it removes the need for extra helper or factory methods just to handle partially available data, and it lets sensible default values be applied automatically for whichever pieces of information the caller chose not to provide.

Part02 – Questions & Answers

Q1: LinkedIn article about abstract class?

A: This item is really a research prompt rather than a single-answer question — it's asking the student to go read an actual article on LinkedIn about abstract classes. Since there's no specific article linked in the task, here's a summary of the points a good article on this topic would typically cover: an abstract class is a class that cannot be instantiated on its own and is meant to be a base for other classes; it can mix fully implemented (concrete) members with abstract members that have no body and must be implemented by any derived class; it's used when a family of related types share both some common behavior and some behavior that must vary per type; and it differs from an interface mainly in that it can hold actual implementation and shared state, at the cost of only allowing single inheritance (a class can extend only one abstract class, but implement many interfaces).

Q2: What we mean by coding against interface rather than class? And if u get it, what we mean by code against abstraction not concreteness?

A: 'Coding against an interface rather than a class' means writing code that depends on a type describing only behavior — a contract — instead of depending on a specific concrete implementation. For example, a method that takes an `IVehicle` parameter can work with a `Car`, a `Bike`, or any future class implementing `IVehicle`, without ever needing to know which one it actually received. 'Coding against abstraction, not concreteness' is the broader, more general version of the same idea, and it isn't limited to interfaces only — it also covers abstract classes, or even a base class used purely for the contract it defines. The underlying principle (closely tied to the Dependency Inversion Principle) is that code should depend on stable, general abstractions — what something can do — rather than on specific, changeable implementation details of exactly how one particular class does it, because abstractions change far less often than implementations, which makes the code more maintainable and far less fragile when implementation details are modified.

Q3: What is abstraction as a guideline and how we can implement this through what we have studied?

A: As a guideline, abstraction means that when designing a class or module, you deliberately decide what its 'public face' should be — exposing only what other code genuinely needs, and hiding every unnecessary implementation detail behind that public face. Through what has been studied in this course, this guideline is put into practice with several concrete C# tools: interfaces (`IVehicle`, `IComparable`, `ILogger`, `IShapeSeries`) define WHAT a type can do without exposing HOW it does it; abstract classes (`Shape`, `GeometricShape`) define a shared abstract contract (`GetArea()`, `CalculateArea()`) while still allowing some shared concrete behavior to live in one place (`Display()`); private fields exposed through public properties (as in `Account` and `Employee`) hide the raw internal data and only expose a controlled way to read or change it; and method overriding through `virtual/abstract/override` lets each concrete class hide its own internal detail behind one shared, abstract signature. Together, these tools let the rest of a program interact with a simple, stable abstraction instead of depending on the messy implementation details underneath — which is exactly the point of treating abstraction as a design guideline rather than just a language feature.

Part03 Bonus – Question

Q: What is operator overloading?

A: Operator overloading lets you redefine how a built-in operator (such as `+`, `-`, `==`, `<`, or `>`) behaves when it's applied to instances of a custom type, by defining a special static method using the operator keyword, for example: `public static Product operator +(Product a, Product b) { ... }`. This allows objects of your own class to be combined or compared using familiar, natural operator syntax instead of calling a specifically named method every time — for instance, adding two `Vector` objects as `vectorA + vectorB`, or comparing two `Money` objects as `moneyA > moneyB`. It should be used carefully and only where the operator's meaning for the type is genuinely intuitive and unambiguous, since overloading operators for operations that don't have an obvious, natural operator-like meaning tends to make code more confusing rather than more readable.

Part03 – Self Study Topics

1. Memory Allocation

Memory in a .NET program is managed mainly in two regions: the stack and the managed heap. The stack is a simple, fast, last-in-first-out region used for local variables and method call frames; memory on the stack is reclaimed automatically and instantly the moment a method returns or a block goes out of scope. The heap is a larger, more flexible region managed by the garbage collector, used to store objects whose lifetime isn't tied to a single method call.

This directly connects to the value type vs. reference type distinction covered throughout this course. Value types (structs, and primitives like `int`, `double`, `bool`) generally live wherever they're declared: on the stack for a local variable, or inline inside their containing object or array if they're a field or an element. Every assignment or method call copies the entire value, producing a fully independent copy. Reference types (classes) always have their actual data allocated on the heap; a variable of a class type only holds a reference (a pointer) to that heap location, so assignment only copies the small reference, leaving both variables pointing at the same underlying object.

This is also why structs tend to be faster for small, short-lived data: no heap allocation happens, and there's nothing left over for the garbage collector to track and eventually clean up. Classes, by contrast, always pay the cost of a heap allocation, but only need to copy a small, fixed-size reference no matter how large the object actually is — which is why classes remain the better choice once an object becomes large, needs shared mutable identity across different parts of a program, or needs to participate in an inheritance hierarchy, something structs cannot do at all.

2. Dependency Inversion Principle (DIP)

The Dependency Inversion Principle is one of the five SOLID principles of object-oriented design. It states that high-level modules (the parts of a program that implement business logic or overall behavior) should not depend directly on low-level modules (the parts that implement specific technical details); instead, both should depend on abstractions. It also states that abstractions should not depend on details — details should depend on abstractions.

In practice, this is exactly what 'coding against an interface rather than a class' achieves. Take `ILogger` and `ConsoleLogger` from this task as an example: instead of a high-level class directly creating and depending on a concrete `ConsoleLogger`, it depends only on the `ILogger` interface, and a `ConsoleLogger` instance is supplied to it from the outside. If a different kind of logger is needed later (a `FileLogger`, or a logger that sends messages to a remote server), the high-level class doesn't need to change at all — only a new class implementing `ILogger` needs to be created and supplied in place of the old one.

This inversion is what gives the principle its name: normally you'd expect a high-level module to directly depend on (and be shaped around) the low-level details it uses, but DIP inverts that relationship so the low-level details depend on an abstraction defined from the high-level module's point of view. This is the same underlying idea behind every interface used across this course's problems (`IVehicle`, `IComparable`, `IWalkable`, `IShapeSeries`) — each one lets the code that uses the interface stay completely unaware of, and unaffected by, whichever concrete implementation happens to be plugged in.

3. Specification Design Pattern (Interface-Based)

The Specification pattern is a design pattern (common in domain-driven design) used to encapsulate a business rule or a piece of selection/validation logic inside its own dedicated object, instead of scattering that logic as raw if-conditions throughout the codebase. It is built almost entirely around the idea of coding against an interface: a common interface, typically something like `ISpecification<T>` with a single method `bool IsSatisfiedBy(T candidate)`, is defined once, and every individual business rule becomes its own small class implementing that interface — for example, an `InStockSpecification`, or a `PriceAboveSpecification` for the `Product` class used earlier in this task.

Because every specification shares the same interface, individual specifications can be combined together using composite specifications (an `AndSpecification`, an `OrSpecification`, a `NotSpecification`) that themselves also implement `ISpecification<T>` and internally call `IsSatisfiedBy` on the specifications they wrap. This lets complex business rules be built up out of small, independently testable, reusable pieces, and lets the exact same rule-checking logic be reused wherever it's needed — for filtering a list, for validating an object before saving it, or for deciding whether an action is currently allowed — without duplicating the underlying condition each time.

This pattern is a direct, practical illustration of everything covered in this course about interfaces and abstraction: the calling code that uses a specification only ever depends on the `ISpecification<T>` interface, never on which concrete rule it actually received, which is precisely the Dependency Inversion idea applied to business rules specifically, rather than to infrastructure components like loggers or data sources.